

RFC: PartFinder — Marketplace de Repuestos

Autores: Efren Felipe Cuadrado Barboza · Juan David Alvarez Garcia · Juan Diego Quintero Ortiz · Roison Garcia Sepulveda · Sebastian Medina Londoño **Materia:** Arquitectura de Software II — 2026-1 **Caso de estudio:** #7 — PartFinder: El Marketplace de Repuestos **Fecha de diseño:** 2026-05-16 **Fecha de cierre de implementación:** 2026-06-01 **Versión:** 1.0 **Estado:** Implementado y verificado end-to-end (R1, R2 y R3 demostrables; observabilidad y bonus Nginx+UI incluidos)

1. Resumen Ejecutivo

El problema

PartFinder conecta talleres mecánicos con bodegas de repuestos. El sistema debe (1) orquestar búsquedas síncronas contra proveedores con tiempos de respuesta heterogéneos, devolviendo un resultado **útil incluso cuando un proveedor es lento**; (2) bloquear pedidos de talleres con crédito excedido; (3) registrar de forma asíncrona qué piezas se buscan sin éxito, para alimentar la inteligencia de inventario de los proveedores.

La solución propuesta

Un ecosistema de **tres microservicios**:

1. **MS-Aggregator** (núcleo hexagonal, Elasticsearch + MySQL secundario) — orquesta búsquedas y pedidos, aloja las 3 reglas del caso.
2. **MS-InventoryDirect** (REST sync, MySQL) — simula las bodegas: responde stock por proveedor, con latencia variable que permite demostrar R1.
3. **MS-TrendCollector** (broker async, PostgreSQL) — consume **SearchFailedEvent**, acumula tendencias por proveedor; nunca está en el camino crítico del usuario.

Comunicación sync (Feign + Resilience4j **@TimeLimiter**) para validación R1, y async (RabbitMQ + Transactional Outbox) para R3. R2 se decide localmente sobre una proyección de crédito dentro de Aggregator.

Resultado esperado

- **GET /search** con latencia p95 < 300 ms cuando InventoryDirect responde rápido.
- **GET /search** con latencia tope **acotada en ~800 ms** incluso cuando InventoryDirect cuelga, devolviendo **availability=UNCERTAIN**.
- **POST /orders** valida R2 en < 50 ms (lectura local).
- 0 búsquedas fallidas perdidas (entrega at-least-once garantizada por Outbox).

2. Atributos de Calidad

Siguiendo la clasificación estándar ISO 25010 vista en el curso (Rendimiento, Seguridad, Usabilidad, Fiabilidad, Mantenibilidad, Escalabilidad, Compatibilidad, Portabilidad), identificamos **tres atributos críticos** para PartFinder.

2.1 Rendimiento / Eficiencia

Definición operacional: `GET /search?query=...` con $p95 < 300$ ms cuando InventoryDirect está sano; cota dura ≤ 800 ms aún cuando InventoryDirect degrada (timeout de Resilience4j).

Mecanismos:

- **Elasticsearch** en Aggregator para búsqueda full-text del catálogo en milisegundos.
- **Proyección local de crédito** en Aggregator para R2 sin llamada remota.
- **Feign + @TimeLimiter (800 ms)**: latencia acotada en el peor caso. El usuario nunca espera más de 800 ms aunque InventoryDirect esté colgado.

2.2 Fiabilidad

Definición operacional: ninguna búsqueda fallida se pierde; el sistema sigue operando con resultados parciales (**UNCERTAIN**) cuando un proveedor degrada.

Mecanismos:

- **Patrón Transactional Outbox** para R3 (entrega at-least-once de `SearchFailedEvent`).
- **Política markUncertain ante timeout (R1)**: degradación elegante en lugar de error.
- **Idempotencia en TrendCollector**: misma búsqueda fallida llegando dos veces no duplica registros.
- **Dead Letter Queues** en la cola de TrendCollector: aislamiento de mensajes problemáticos.

2.3 Mantenibilidad

Definición operacional: cambios típicos a una regla de negocio tocan ≤ 2 clases.

Mecanismos:

- **Hexagonal estricto** — dominio sin frameworks; tests del dominio sin Spring, sub-segundo.
- **Principios SOLID** sistemáticamente aplicados.
- **4 patrones GoF** (Strategy, Adapter, Factory, Observer).
- **Migraciones de schema versionadas con Flyway** donde hay store relacional.

3. Decisiones Arquitectónicas

3.1 Lenguaje y framework

Decisión	Java 25 LTS + Spring Boot 3.5
Alternativas evaluadas	Node.js/NestJS, Python/FastAPI, .NET 8
Justificación	(1) Recomendado por el profesor. (2) Ecosistema maduro (Feign, Spring AMQP, Spring Data Elasticsearch, springdoc, Micrometer, OTel). (3) Experiencia del equipo.

3.2 Persistencia políglota

MS	DB primaria	DB secundaria (si aplica)	Justificación
Aggregator	Elasticsearch 8	MySQL 8 (solo para workshops + outbox_events)	ES brilla en búsqueda full-text del catálogo con scoring; MySQL provee ACID para Outbox y proyección de crédito. La rúbrica permite polyglot dentro de un MS (la prohibición aplica a sharing entre MS).
InventoryDirect	MySQL 8	—	Datos relacionales simples, modelo transaccional.
TrendCollector	PostgreSQL 16	—	Tipos avanzados (JSONB para auditoría flexible), agregaciones analíticas vía window functions.

3.3 Comunicación inter-servicios

Tipo	Tecnología	Cuándo se usa
Sync	OpenFeign + Resilience4j @TimeLimiter (800 ms)	Aggregator → InventoryDirect, durante cada GET /search . El TimeLimiter materializa R1.
Async	RabbitMQ 3.13 + Spring AMQP	Aggregator → TrendCollector (eventos SearchFailedEvent). Una sola dirección.

RabbitMQ sobre Kafka: simplicidad operacional, UI de management embebida, throughput suficiente.

3.4 Arquitectura interna de cada MS

Arquitectura Hexagonal (Ports & Adapters) con **domain/**, **application/**, **infrastructure/**. Regla de dependencias hacia adentro. El dominio no conoce frameworks.

3.5 Patrones de diseño aplicados

Patrón	Categoría	Dónde
Strategy	Comportamiento	OrderCreditPolicy (R2), composición de SearchPolicy en SearchPartUseCase
Adapter	Estructural	InventoryDirectFeignAdapter , ElasticsearchPartRepositoryAdapter , RabbitSearchFailedPublisher
Factory	Creacional	SearchResultFactory (encapsula construcción con availability correcto)
Observer / Pub-Sub	Comportamiento	DomainEventPublisher + Outbox + RabbitMQ
Transactional Outbox	Integración (no GoF)	Garantía de entrega de eventos para R3

Patrón	Categoría	Dónde
TimeLimiter + Circuit Breaker (Resilience4j)	Resiliencia	R1 — degradación elegante a UNCERTAIN

3.6 Observabilidad

Stack abierto: **Prometheus** (métricas), **Grafana** (dashboards), **Jaeger** (trazas distribuidas), vía **Micrometer** y **OpenTelemetry Java Agent**.

3.7 Bonificaciones

- **Nginx** como API Gateway: mismo container que sirve la UI estática (un proceso, menos superficie).
- **React 18 + Vite** como UI: SPA con búsqueda, creación de pedidos, vista de tendencias y panel admin para forzar slow-response en InventoryDirect (demo de R1).

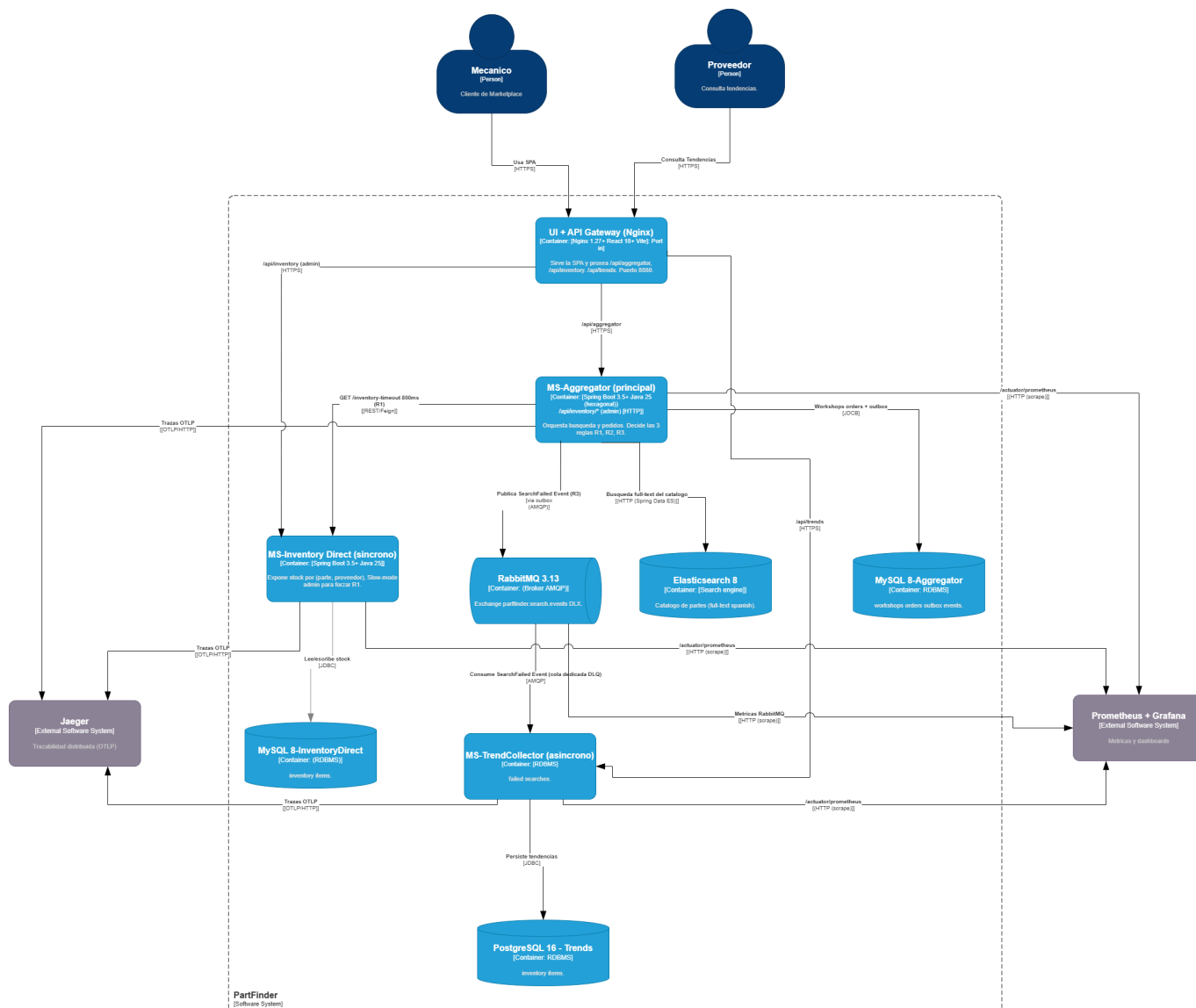
4. Trade-offs

#	Decisión	Beneficio	Costo aceptado
1	Polyglot dentro de Aggregator (ES + MySQL)	ES para búsqueda + MySQL para ACID donde se necesita	Más complejidad operacional dentro del mismo MS; dos drivers
2	R1 degradación a UNCERTAIN en lugar de rechazo	El usuario sigue recibiendo respuesta útil	La UI debe distinguir UNCERTAIN visualmente; experiencia más rica pero más diseño
3	Transactional Outbox para R3	Entrega garantizada sin 2PC ni Sagas	Una tabla extra + un worker; latencia de propagación de hasta 1 s
4	Proyección local de crédito	Validación R2 sub-ms; sin acoplar Aggregator a otro MS	Stale reads si el cupo cambió en los últimos segundos (mitigable)
5	El "MS financiero" como agregado dentro de Aggregator	No explosión de servicios; rúbrica solo pide 3 MS	Trade-off conceptual: en producción real estaría separado
6	RabbitMQ sobre Kafka	Simplicidad operacional, UI embebida	Sin event sourcing ni replays masivos (no los necesitamos)
7	Hexagonal estricto	Tests del dominio sin Spring; reemplazo de adaptadores trivial	Más carpetas y clases que un CRUD plano

Trade-off central de PartFinder:

Sacrificamos **respuesta determinista** (sí/no rotundo) a cambio de **disponibilidad útil** (incluso bajo timeout damos información parcial). Esto es la materialización directa de la regla R1 del enunciado: el negocio prefiere "incierto pero pronto" antes que "perfecto pero tarde o caído".

5. Diagrama C4 Nivel 2 (Contenedores)



Fuente editable: [diagrams/c4/c4-L2-containers.drawio](#) (draw.io). El diagrama de contexto (C4 Nivel 1) está en el mismo directorio ([c4-L1-context.puml](#) / [.png](#)).

Resumen narrativo del diagrama L2

- El **mecánico/usuario del taller** accede a la plataforma a través de la **UI Web** (React).
- La UI llama al **API Gateway (Nginx)** que enruta las peticiones a los microservicios correspondientes.
- MS-Aggregator** es el núcleo: recibe `/search` y `/orders`, orquesta las validaciones.
 - Llama síncronamente a **MS-InventoryDirect** vía Feign con timeout 800 ms para validar R1.
 - Lee de su propia DB (MySQL secundario) la proyección de crédito para validar R2.
 - Publica eventos `SearchFailedEvent` al broker RabbitMQ vía Outbox para R3.
 - Persiste catálogo de partes y pedidos en Elasticsearch + MySQL.
- MS-InventoryDirect** lee stock de MySQL y responde (con latencia simulable para demo).
- MS-TrendCollector** consume `SearchFailedEvent`, agrega en PostgreSQL y expone `/trends` para análisis.
- Los tres MS exponen métricas a **Prometheus** vía `/actuator/prometheus` y trazas a **Jaeger** vía OpenTelemetry. **Grafana** visualiza ambas fuentes.

6. Verificación end-to-end del sistema implementado

Esta sección documenta lo que efectivamente se construyó y se verificó funcionando.

6.1 Cobertura de mínimos de rúbrica

Mínimo de rúbrica	Cumplimiento
Arquitectura hexagonal estricta en los 3 MS	✓ <code>domain/</code> , <code>application/</code> , <code>infrastructure/</code> en cada servicio. El dominio no importa Spring ni JPA.
Base de datos propia por servicio	✓ Elasticsearch + MySQL (Aggregator, polyglot intra-MS), MySQL (InventoryDirect), PostgreSQL (TrendCollector). Ninguna se comparte entre MS.
Principios SOLID demostrables	✓ Aplicados; visibles en la capa de dominio (<code>domain/</code>) y en la inversión de dependencias por puertos/adaptadores (p. ej. <code>OrderRulePolicy</code> como abstracción que <code>CreateOrderUseCase</code> consume sin conocer la implementación).
Mínimo 3 patrones GoF	✓ Implementados los 4: Strategy (<code>OrderCreditPolicy</code>), Adapter (Feign/ES/JPA/AMQP), Factory (<code>SearchResultFactory</code>), Observer (<code>DomainEventPublisher</code> + Outbox).
Validación programática de las 3 reglas del caso en la capa de dominio	✓ R1 en <code>domain/factory/SearchResultFactory.java</code> + <code>infrastructure/client/InventoryDirectFeignAdapter.java</code> ; R2 en <code>domain/policy/OrderCreditPolicy.java</code> ; R3 mediante outbox transaccional desde <code>application/usecase/SearchPartUseCase.java</code> .
Swagger/OpenAPI funcional por MS	✓ springdoc-openapi 2.8 en los 3 MS, expuesto en <code>/swagger-ui.html</code> .
1 MS Principal + 1 MS Síncrono REST + 1 MS Asíncrono Broker	✓ Aggregator (core hexagonal) + InventoryDirect (REST sync) + TrendCollector (broker async).
Comunicación síncrona vía cliente REST declarativo	✓ OpenFeign + Spring Cloud, instrumentado con Resilience4j CircuitBreaker + bean <code>Request.Options</code> con <code>readTimeout=800ms</code> .
Comunicación asíncrona vía broker	✓ RabbitMQ 3.13 con exchange topic <code>partfinder.search.events</code> , DLX <code>partfinder.trends.dlx</code> y DLQ.
Docker Compose orquesta todo con un solo comando	✓ <code>docker compose up -d --build</code> levanta 12 contenedores.
Stack de observabilidad (Prometheus + Grafana + Jaeger)	✓ Auto-provisionado: datasources Prometheus + Jaeger en Grafana, dashboard "PartFinder Health" con 6 paneles, OTel Agent en los 3 MS exportando vía OTLP.

Mínimo de rúbrica	Cumplimiento
Documento RFC y Modelo C4 Nivel 2	✓ Este documento + diagramas PlantUML en docs/diagrams/c4/ .
Repositorio Git público con tabla de entregables en el README	✓ https://github.com/SebasMedina22/arquitectura-2-partfinder

6.2 Bonificaciones implementadas

Bonificación	Cumplimiento
API Gateway	✓ Nginx 1.27 en contenedor ui-gateway haciendo proxy reverso de los tres prefijos /api/aggregator , /api/inventory , /api/trends .
Interfaz de Usuario	✓ SPA React 18 + Vite servida por el mismo Nginx (puerto host 8080). Cinco pestañas: Buscar parte, Crear pedido, Pedidos del taller, Tendencias, Admin de simulación.

6.3 Demostraciones funcionales verificadas

Escenario	Solicitud	Resultado
Camino feliz	GET /search?query=filtro&workshopId=WS-001	HTTP 200, availability=AVAILABLE , 3 proveedores con stock
R1 — degradación por timeout	slow-mode=1500ms en InventoryDirect, buscar de nuevo	HTTP 200, latencia ≈ 1216ms (cota dura por readTimeout=800ms + roundtrip), availability=UNCERTAIN
R1 — Circuit Breaker	InventoryDirect cae > 50% en ventana de 10	Breaker abre, fallback markUncertain sin tocar la red
R2 — bloqueo por crédito	POST /orders con workshopId=WS-003 (cupo excedido en seed)	HTTP 422, código R2_CREDIT_EXCEEDED
R3 — búsqueda fallida	GET /search?query=xyz-no-existe	HTTP 200 [] ; evento en outbox_events ; worker lo publica en <1s; TrendCollector lo guarda; aparece en GET /trends/top
Idempotencia consumer	Reintento del mismo SearchFailedEvent	PRIMARY KEY event_id rechaza el duplicado
Trazabilidad distribuida	GET /search desde la UI	Jaeger muestra trace con ~12 spans cubriendo ui-gateway → ms-aggregator → Feign → ms-inventory-direct → MySQL con mismo trace_id

6.4 Decisiones revisadas durante implementación

1. **Polyglot intra-MS en Aggregator** (ES + MySQL juntos): ES para búsqueda, MySQL para Outbox + proyección de crédito. La rúbrica permite polyglot dentro de un MS.
 2. **Puertos del compose elegidos para evitar colisiones con otros stacks locales** (p. ej. MySQL en 3307/3308, PostgreSQL en 5434). No correr dos stacks con los mismos puertos a la vez.
 3. **Configuración de Feign timeouts vía bean `Request.Options`**. Spring Cloud 2025.x cambió el namespace de la propiedad — el bean explícito es determinístico y resistente a cambios futuros.
 4. **Topología AMQP creada en runtime** por Spring AMQP. No usamos `load_definitions` porque RabbitMQ ignora `RABBITMQ_DEFAULT_USER/PASS` cuando hay un definitions montado.
 5. **API Gateway y UI fusionados en un solo contenedor `ui-gateway`**.
-

Anexos

- [README.md](#) — Tabla de entregables, stack, instrucciones de despliegue.
- [docs/diagrams/c4/](#) — Diagramas C4 Nivel 1 (Contexto) y Nivel 2 (Contenedores) en PlantUML y PNG.